

Fundamentals of Deep Learning and Programming

Lecture 9: Transformer (1)

Fall Semester 2025

Adjunct Professor: Donghoon Kang

Term Project

Due date: Two weeks later

Week 15 (12.12, Friday)

Should **submit** an **5 ~ 6 page report** as a term-project.

December						
Su	Mo	Tu	We	Th	Fr	Sa
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

Matrix Form: Column Vector Convention

In many mathematical treatments of neural networks, the linear transformation is expressed using column vectors:

$$y = Wx + b,$$

where

- x is an input column vector,
- W is the weight matrix,
- b is the bias vector.

This is the traditional notation used in linear algebra and theoretical descriptions of neural networks.

PyTorch Implementation: Row-Major Storage

PyTorch stores the weight matrix in a row-major layout for efficiency and consistency. In particular:

$$W \in \mathbb{R}^{(\text{out_features}) \times (\text{in_features})}.$$

Given a batch of input vectors arranged in a matrix:

$$x \in \mathbb{R}^{(\text{batch}) \times (\text{in_features})},$$

PyTorch performs the linear transformation as:

$$y = xW^T + b.$$

This ensures that:

$$x \in \mathbb{R}^{(B \times I)}, \quad W^T \in \mathbb{R}^{(I \times O)} \quad \Rightarrow \quad xW^T \in \mathbb{R}^{(B \times O)},$$

which matches the expected output shape.

Although the mathematical form is commonly written as:

$$y = Wx + b,$$

PyTorch implements the computation as:

$$y = xW^T + b,$$

which is equivalent but adapted for efficient batch computation and memory layout.

nn.Linear in PyTorch

```
import torch
import torch.nn as nn

layer = nn.Linear(in_features=4, out_features=3)

# y = x @ layer.weight.T + layer.bias
# x: input
# y: output
print(f"layer.weight : {layer.weight}") # weight shape = (3, 4)
print(f"layer.bias   : {layer.bias}") # bias shape = (3,)
```

PyTorch performs the linear transformation as:

$$y = xW^T + b.$$

This ensures that:

$$x \in \mathbb{R}^{(B \times I)}, \quad W^T \in \mathbb{R}^{(I \times O)} \quad \Rightarrow \quad xW^T \in \mathbb{R}^{(B \times O)},$$

which matches the expected output shape.

torch.arange() in PyTorch

```
import torch

a = torch.arange(5)
print(a)

b = torch.arange(2, 10)
print(b)

c = torch.arange(2, 10, 2)
print(c)
```

```
tensor([0, 1, 2, 3, 4])
tensor([2, 3, 4, 5, 6, 7, 8, 9])
tensor([2, 4, 6, 8])
```

`torch.arange(start, end, step, dtype, device)`

- start (optional) — first value (default: 0)
- end — stops before this value
- step (optional) — spacing between values (default 1)
- dtype (optional) — data type (e.g., torch.float32)
- device (optional) — CPU/GPU location (default: CPU)

```
import torch

d = torch.arange(5, device='cuda')
print(d)

e = torch.arange(5, device='cpu')
print(e)
```

```
tensor([0, 1, 2, 3, 4], device='cuda:0')
tensor([0, 1, 2, 3, 4])
```

unsqueeze() in PyTorch

```
import torch
```

```
row = torch.arange(3).unsqueeze(0) # shape (1,3)  
col = torch.arange(3).unsqueeze(1) # shape (3,1)
```

```
print("row: ", row)  
print("col: ", col)
```

```
row: tensor([[0, 1, 2]])  
col: tensor([[0],  
            [1],  
            [2]])
```

```
# element-wise multiplication
```

```
print("col*row : ", col*row) # broadcast  
print("row*col : ", row*col) # broadcast
```

```
col*row : tensor([[0, 0, 0],  
                [0, 1, 2],  
                [0, 2, 4]])  
row*col : tensor([[0, 0, 0],  
                [0, 1, 2],  
                [0, 2, 4]])
```

torch.arange does not create matrices.

To expand dimension, we must
use `.unsqueeze()` (supports `broadcasting`)

```
col --> [[0, 0, 0],  
         [1, 1, 1],  
         [2, 2, 2]]
```

```
row --> [[0, 1, 2],  
         [0, 1, 2],  
         [0, 1, 2]]
```

Both become (3,3).

unsqueeze() in PyTorch

```
import torch

# A simple 1D tensor (vector)
x = torch.tensor([1.0, 2.0, 3.0]) # shape: (3,)
print("x * x", x * x)
print("x @ x", x @ x) # torch.dot(x,x)

x * x tensor([1., 4., 9.])
x @ x tensor(14.)

# Two different unsqueeze operations:
row = x.unsqueeze(0) # shape: (1, 3)
col = x.unsqueeze(1) # shape: (3, 1)

# element-wise multiplication (broadcasting is required)
print("col*row : ", col*row) # broadcast to (3,3)*(3,3) = (3,3)
print("row*col : ", row*col) # broadcast to (3,3)*(3,3) = (3,3)

# matrix multiplication
print("col@row : ", col@row) # (3,1) @ (1,3) = (3,3)
print("row@col : ", row@col) # (1,3) @ (3,1) = (1,1)
```

```
col*row : tensor([[0, 0, 0],
                  [0, 1, 2],
                  [0, 2, 4]])
row*col : tensor([[0, 0, 0],
                  [0, 1, 2],
                  [0, 2, 4]])
```

```
col@row : tensor([[0, 0, 0],
                  [0, 1, 2],
                  [0, 2, 4]])
row@col : tensor([[5]])
```

register_buffer() in nn.Module

```
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(3))
        self.register_buffer("scale", torch.tensor(2.0))

model = MyModel()
print (model.state_dict())
```

```
OrderedDict([('weight', tensor([-0.9091, -0.1381,  0.6516])), ('scale', tensor(2.))])
```

In PyTorch, `register_buffer(name, tensor)` is a method of `nn.Module` that attaches a tensor to the module without treating it as a learnable parameter.

A buffer is a tensor that:

- belongs to the module,
- is saved and loaded with `state_dict()`,
- moves automatically with `model.to(device)`,
- is **not** updated by the optimizer.

Parameters vs. Buffer in nn.Module

	Stored in model	Updated by optimizer	Moves with <code>to(device)</code>	In <code>state_dict()</code>
Parameter	Yes	Yes	Yes	Yes
Buffer	Yes	No	Yes	Yes

A buffer is ideal for tensors that are part of the model but should not be trained.

Why Use Buffer ?

Some tensors should remain part of the model's state but never receive gradients.

Examples include:

- sinusoidal positional encodings,
- BatchNorm running statistics,
- constant masks,
- fixed lookup tables,
- normalization constants.

Why do we need state_dict() ?

```
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(3))
        self.register_buffer("scale", torch.tensor(2.0))

model = MyModel()
print (model.state_dict())
```

```
model.state_dict()
```

returns a dictionary similar to:

```
{
    'weight': tensor([...]),
    'scale': tensor(2.)
}
```

```
OrderedDict([('weight', tensor([-0.9091, -0.1381,  0.6516])), ('scale', tensor(2.))])
```

In PyTorch, `model.state_dict()` returns a Python dictionary containing the complete internal state of a model. It includes all trainable parameters and all registered buffers. This dictionary represents the essential data required to save or load a model reliably.

What does `state_dict()` contains ?

(1) Trainable Parameters

These are tensors wrapped in `nn.Parameter`, such as:

- weights,
- biases,
- linear layer matrices,
- convolution kernels.

(2) Registered Buffers

Buffers are persistent tensors registered using:

```
self.register_buffer("name", tensor)
```

Examples include:

- sinusoidal positional encodings,
- BatchNorm running mean and variance,
- fixed masks,
- lookup tables or constants.

Save and Load Models using PyTorch

```
import torch
import torch.nn as nn

# -----
# Simple model definition
# -----
class SimpleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(4, 3)

    def forward(self, x):
        return self.linear(x)

# -----
# 1. Create a model and inspect initial weights
# -----
model = SimpleModel()
print("Initial model weights:")
print(model.linear.weight)
print()
```

```
Initial model weights:
Parameter containing:
tensor([[ 0.2515, -0.3357,  0.1317,  0.3983],
        [-0.4345,  0.2308, -0.3156,  0.4657],
        [ 0.0736, -0.1300,  0.4589, -0.1545]], requires_grad=True)
```

Save and Load Models using PyTorch

```
# -----  
# 2. Save model weights to a file  
# -----  
torch.save(model.state_dict(), "model.pth")  
print("Model saved to model.pth")  
print()  
  
# -----  
# 3. Create a NEW model instance (untrained / random weights)  
# -----  
new_model = SimpleModel()  
print("New model BEFORE loading:")  
print(new_model.linear.weight)  
print()
```

Model saved to model.pth

New model BEFORE loading:

Parameter containing:

```
tensor([[ -0.1537,  0.1422,  0.1379,  0.1984],  
        [ -0.3955, -0.4262,  0.2577, -0.4344],  
        [ 0.1168,  0.0167,  0.1256,  0.3544]], requires_grad=True)
```

Save and Load Models using PyTorch

```
# -----  
# 4. Load the saved weights into the new model  
# -----  
state_dict = torch.load("model.pth", weights_only=True)  
new_model.load_state_dict(state_dict)  
  
print("New model AFTER loading saved weights:")  
print(new_model.linear.weight)  
print()  
  
# -----  
# 5. Verify the two models now match  
# -----  
print("Do the weights match?", torch.allclose(model.linear.weight, new_model.linear.weight))
```

```
New model AFTER loading saved weights:  
Parameter containing:  
tensor([[ 0.2515, -0.3357,  0.1317,  0.3983],  
        [-0.4345,  0.2308, -0.3156,  0.4657],  
        [ 0.0736, -0.1300,  0.4589, -0.1545]], requires_grad=True)  
  
Do the weights match? True
```


torch.allclose() in PyTorch

```
import torch
import torch.nn as nn

a = torch.tensor([1.0])
b = torch.tensor([1.0 + 1e-7])

print(torch.equal(a, b))      # False
print(torch.allclose(a, b))  # True
```

Common use case: **verifying model loading**

torch.Tensor.masked_fill() in PyTorch

```
import torch

x = torch.tensor([1.0, 2.0, 3.0, 4.0])

# Create a boolean mask
mask = torch.tensor([False, True, False, True])

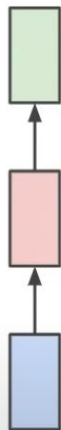
# Use masked_fill
y = x.masked_fill(mask, -5.0) # elements with True in the mask are replaced by -5.0

print("Original tensor:", x)
print("Mask:", mask)
print("After masked_fill:", y)
```

```
Original tensor: tensor([1., 2., 3., 4.])
Mask: tensor([False,  True, False,  True])
After masked_fill: tensor([ 1., -5.,  3., -5.])
```

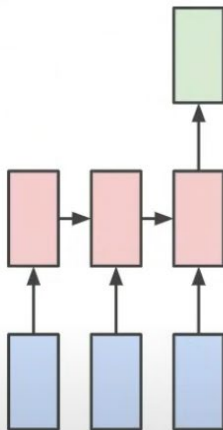
Sequential Data

One-to-one



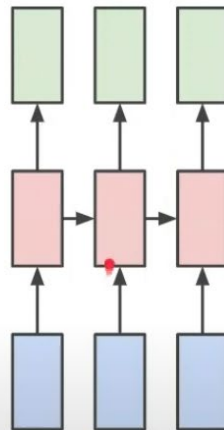
Vanilla Neural-net
Image classification

Many-to-one



Action recognition
(Frames $\rightarrow$ class)

Many-to-many



Frame classification
(Frames $\rightarrow$ classes)

One-to-many

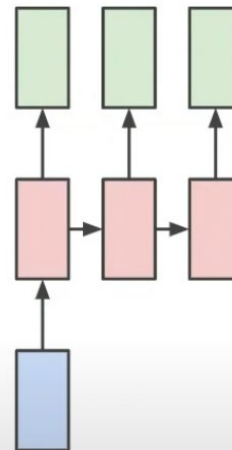
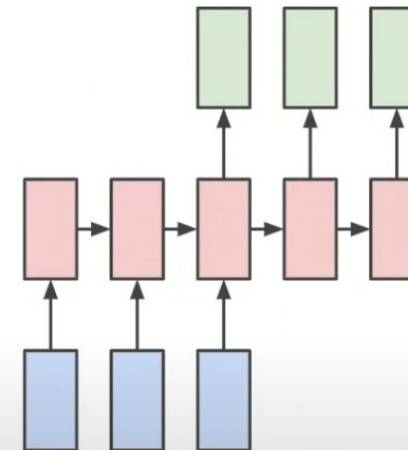


Image captioning
(Image $\rightarrow$ words)

Many-to-many

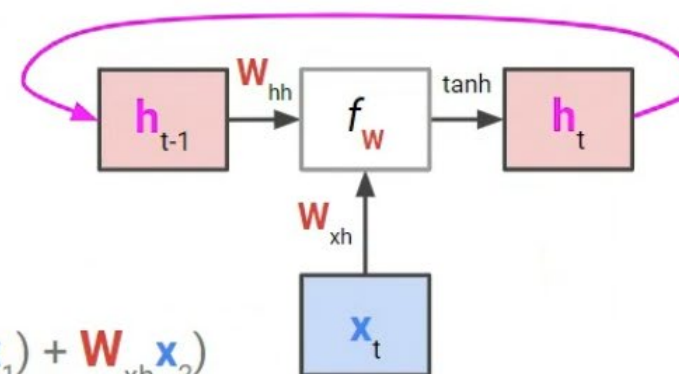


Video captioning
(Video $\rightarrow$ words)

RNN (Recurrent Neural Network)

Then, what about RNNs?

$$\mathbf{h}_t = f_{\mathbf{w}}(\mathbf{h}_{t-1}, \mathbf{x}_t) = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t)$$



$$\mathbf{h}_1 = \tanh(\mathbf{W}_{hh} \mathbf{h}_0 + \mathbf{W}_{xh} \mathbf{x}_1)$$

$$\mathbf{h}_2 = \tanh(\mathbf{W}_{hh} \mathbf{h}_1 + \mathbf{W}_{xh} \mathbf{x}_2) = \tanh(\mathbf{W}_{hh} \tanh(\mathbf{W}_{hh} \mathbf{h}_0 + \mathbf{W}_{xh} \mathbf{x}_1) + \mathbf{W}_{xh} \mathbf{x}_2)$$

$$\begin{aligned} \mathbf{h}_3 &= \tanh(\mathbf{W}_{hh} \mathbf{h}_2 + \mathbf{W}_{xh} \mathbf{x}_3) \\ &= \tanh(\mathbf{W}_{hh} \tanh(\mathbf{W}_{hh} \tanh(\mathbf{W}_{hh} \mathbf{h}_0 + \mathbf{W}_{xh} \mathbf{x}_1) + \mathbf{W}_{xh} \mathbf{x}_2) + \mathbf{W}_{xh} \mathbf{x}_3) \end{aligned}$$

Again, the output $\hat{\mathbf{y}}$ is a weighted sum (+fixed unary operations) of input $\mathbf{x}$!

What is Transformer ?

A **Transformer** is a neural network architecture introduced in the 2017 paper “*Attention Is All You Need*” by Vaswani et al. It was designed for sequence-to-sequence tasks such as machine translation, and it has since become the foundation of most modern language models (e.g. BERT, GPT) and many vision models.

The key idea is to replace recurrent or convolutional operations with a mechanism called **self-attention**.

Transformer: Main Idea

- Basic assumption: the input x can be split into **multiple elements** that are **organically related** to each other.
 - People in a society
 - Words in a sentence
 - Frames in a video
 - ...
- **Self-attention**: Each element learns to refine its own representation by attending its **context** (other elements in the input).
 - More specifically, as a **weighted sum** of **other elements** (not arbitrary weights) in the sequence.

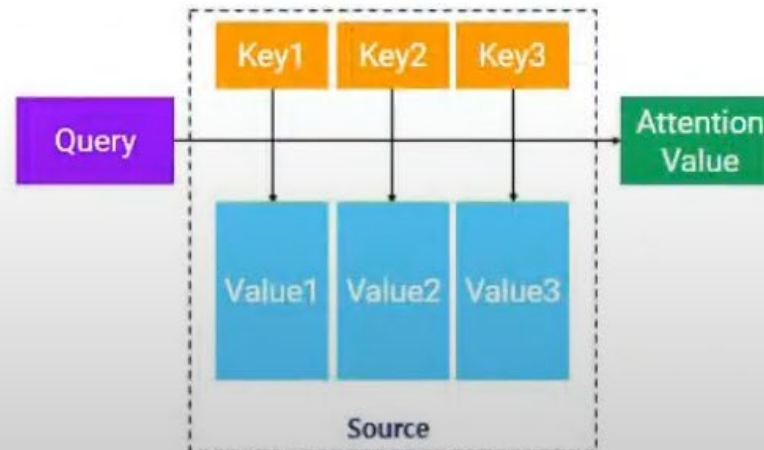


Attention (1)

- **Attention function:** $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Attention value}$

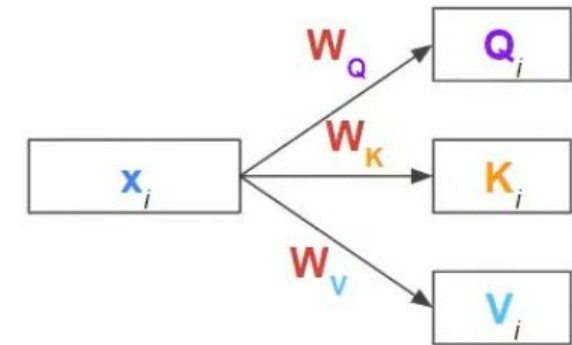
For a **query** (context) and **key-value** pairs (references), **attention value** is the weighted average of values, where each weight is proportional to the relevance between the query and the corresponding key.

- **Q** and **K** must be comparable (usually in the same dimensionality).
- **V** and **Attention value** are in the same dimensionality, obviously.
- In many applications, all four of these are in same dimensionality.



Attention (2)

- So, what should be the **Query**, **Key**, and **Value**?
- With Transformer, we **make** them!
 - From the input tokens $\{x_1, x_2, \dots, x_N\}$,
 - Each token x_i is mapped to its own **Query** Q_i , **Key** K_i , **Value** V_i vectors by a **linear transformation**.
 - The linear weights (W_Q, W_K, W_V) are the **learned parameters**, **shared by all inputs**.
 - W_Q (W_K, W_V) learns how to represent a vector to serve as a **Query** (**Key**, **Value**) in general.
- We need another learnable parameter, W_O , which maps the **attention value** back to the original space.



<https://arxiv.org/pdf/1706.03762.pdf>

Attention (3)

- Then, how do we perform attention?
 - Each token $\mathbf{x}_i$ becomes the **Query** when we learn about i .
 - *You are the main character in your life!*
 - References are all tokens $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ in the input sequence, including $\mathbf{x}_i$ itself.
 - *Your friends are a mirror that reflects you.*
- From this, we perform the attention:
 - Each element $\mathbf{x}_i$ is represented as a weighted (similarity computed using **Key**) sum of other elements (using **Value**) in $\mathbf{x}$.
 - $\mathbf{z}_i = \mathbf{w}_1 \mathbf{V}_1 + \dots + \mathbf{w}_N \mathbf{V}_N$, where $\mathbf{w}_j = \cos(\mathbf{Q}_i, \mathbf{K}_j)$
 - $\mathbf{W}_o$ maps from the **Value** space back to the original embedding space.

What is Transformer ?

2. Self-Attention and Scaled Dot-Product Attention

2.1. Input Embeddings

Assume we have an input sequence of T tokens. Each token is mapped to a vector of dimension d_{model} . We can stack these into a matrix:

$$X \in \mathbb{R}^{T \times d_{\text{model}}}.$$

From X , the Transformer computes three matrices:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V,$$

where

$$W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_k}.$$

Here,

- $Q = \text{queries}$
- $K = \text{keys}$
- $V = \text{values}$

What is Transformer ?

2.2. Scaled Dot-Product Attention

The **scaled dot-product attention** computes a weighted sum of the value vectors. For a set of queries Q , keys K , and values V , attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V.$$

Explanation:

- $QK^{\top}$ computes all pairwise dot products between queries and keys.
- The division by $\sqrt{d_k}$ is the *scaling*; it keeps the magnitude of the dot products from growing too large when d_k is large.
- softmax turns each row into a probability distribution over all positions.
- Multiplying by V computes a weighted sum of value vectors for each query.

This allows each token to attend to (i.e. focus on) other tokens in the sequence.

What is Transformer ?

3. Multi-Head Attention

Instead of using a single attention mechanism, Transformers use **multiple heads**.

Each head has its own projection matrices:

$$Q_h = XW_h^Q, \quad K_h = XW_h^K, \quad V_h = XW_h^V,$$

and produces:

$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h).$$

Then all heads are concatenated:

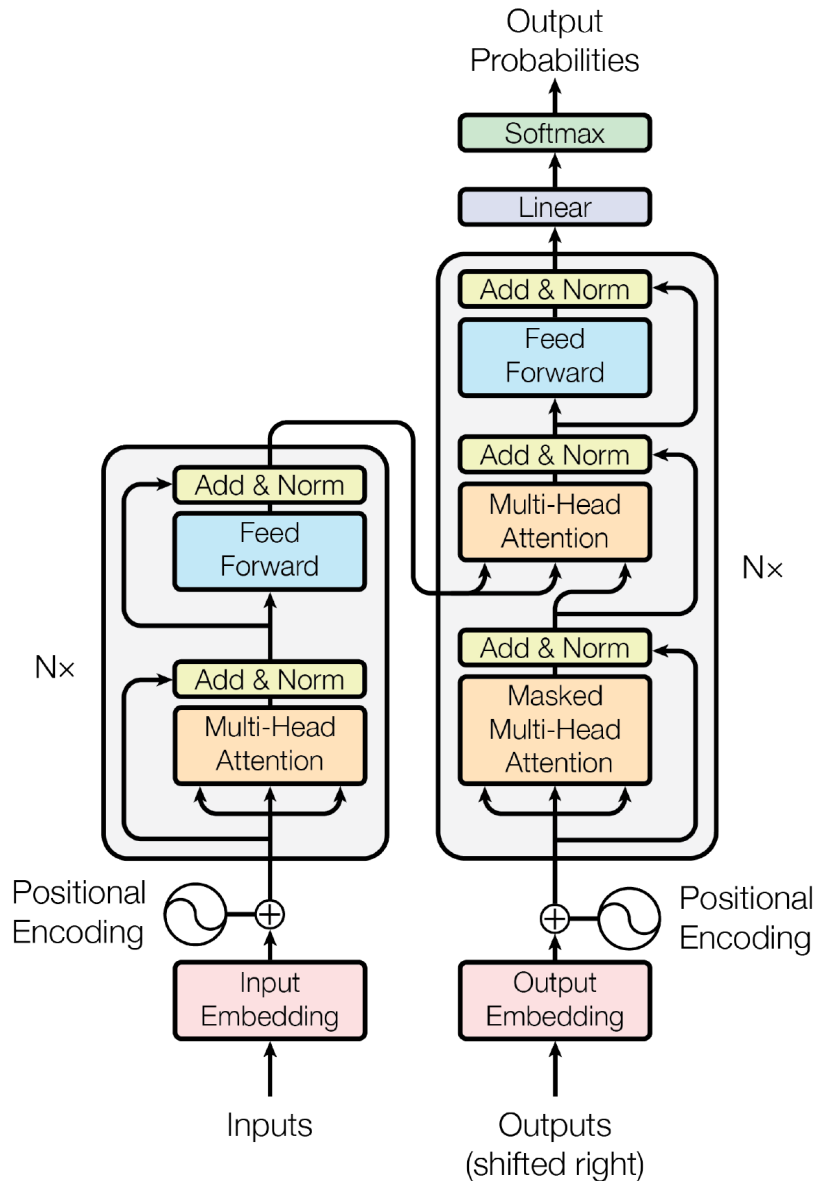
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W^O,$$

where W^O is another learned projection matrix.

Intuition:

- Each head can focus on different types of relationships (e.g. syntax, long-range dependencies).
- Concatenating them allows the model to combine these different perspectives.

Transformer model architecture



"Attention is All you Need" (<https://arxiv.org/abs/1706.03762>)
NIPS 2017

Why Transformer use Positional Encoding ?

Because Transformers do not process tokens sequentially like RNNs, they need a way to represent the order of tokens. They add **positional encodings** P to the input embeddings:

$$Z = X + P.$$

The original paper uses sinusoidal functions for positional encodings, so that the model can infer relative positions.

Positional encoding is designed to **inject** information about the **sequence position** into token embeddings.

Smooth and Continuous Encoding

The sinusoidal positional encoding uses a combination of sine and cosine functions of varying frequencies. For a token at position pos within the sequence and for dimension i of the embedding vector, the encoding is defined as:

$$\begin{aligned} \text{PE}(pos, 2i) &= \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \\ \text{PE}(pos, 2i + 1) &= \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \end{aligned}$$

where:

- pos is the position of the token (from 0 to $\text{max_len} - 1$).
- i is the dimension index (from 0 to $d_{\text{model}}/2 - 1$).
- d_{model} is the dimensionality of the embedding.

Because sine and cosine are smooth and continuous, nearby positions have similar encodings. High-frequency components capture short-range patterns, while low-frequency components capture long-range dependencies.

Understanding Sinusoidal Positional Encoding

A key identity used in understanding sinusoidal positional encoding is:

$$\cos(a - b) = \cos a \cos b + \sin a \sin b,$$

This identity shows that $\cos(a - b)$ can be computed using only sine and cosine values of a and b .

We can define the positional encoding as follows: For a frequency ω and position pos :

$$PE(pos) = \begin{bmatrix} \cos(\omega \cdot pos) \\ \sin(\omega \cdot pos) \end{bmatrix}.$$

Thus, two positions pos_1 and pos_2 are encoded as:

$$PE(pos_1) = \begin{bmatrix} \cos(\omega pos_1) \\ \sin(\omega pos_1) \end{bmatrix}, \quad PE(pos_2) = \begin{bmatrix} \cos(\omega pos_2) \\ \sin(\omega pos_2) \end{bmatrix}.$$

Why This Helps Self-Attention ?

Self-attention computes similarity scores using dot products:

$$\text{score}(i, j) = Q_i \cdot K_j.$$

When positional encodings are added to token embeddings, the sinusoidal structure ensures that the dot products between queries and keys contain information about:

$$pos_i - pos_j,$$

allowing the model to infer how far apart two positions are.

This happens automatically due to the mathematical properties of sine and cosine.

- The dot product between encodings depends only on relative position differences.
- Self-attention readily exploits this because it is based on dot products.

Dot Product Encodes Relative Distance

Compute the dot product:

$$PE(pos_1) \cdot PE(pos_2) = \cos(\omega pos_1) \cos(\omega pos_2) + \sin(\omega pos_1) \sin(\omega pos_2).$$

Using the identity:

$$\cos(x - y) = \cos x \cos y + \sin x \sin y,$$

we obtain:

$$PE(pos_1) \cdot PE(pos_2) = \cos(\omega(pos_1 - pos_2)).$$

Important Observation: The dot product depends only on $(pos_1 - pos_2)$, meaning that sinusoidal encodings naturally encode **relative position information**.

Understanding Sinusoidal Positional Encoding

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$
$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

where:

- pos is the position of the token (from 0 to $\text{max_len} - 1$).
- i is the dimension index (from 0 to $d_{\text{model}}/2 - 1$).
- d_{model} is the dimensionality of the embedding.

The term $10000^{2i/d_{\text{model}}}$ is equivalent to $\exp(\frac{2i}{d_{\text{model}}} \cdot \ln(10000))$, which is implemented in the code as $\exp(\frac{2i}{d_{\text{model}}} \cdot \ln(10000.0))$. The code computes the reciprocal of the denominator's base:

$$\text{div_term} = \exp\left(\frac{2i}{d_{\text{model}}} \cdot (-\ln(10000.0))\right) = \frac{1}{10000^{2i/d_{\text{model}}}}$$

This `div_term` is then multiplied by the position vector, effectively computing the argument of the sine and cosine functions.

```
# frequency term for even dimensions
div_term = torch.exp(
    torch.arange(0, d_model, 2, dtype=torch.float32)
    * (-math.log(10000.0) / d_model)
)
```

Positional Encoding (1) : separate heatmaps for sin and cos

```
import torch
import math
import matplotlib.pyplot as plt

# =====
# Generate sinusoidal positional encoding
# =====

def generate_positional_encoding(d_model, max_len=200):

    # pe shape: (max_len, d_model)
    pe = torch.zeros(max_len, d_model)

    # positions: 0 .. max_len-1, shape = (max_len, 1)
    position = torch.arange(0, max_len, dtype=torch.float32).unsqueeze(1)

    # frequency term for even dimensions
    div_term = torch.exp(
        torch.arange(0, d_model, 2, dtype=torch.float32)
        * (-math.log(10000.0) / d_model)
    )

    # Even indices: sin, shape broadcasting: (max_len, 1) * (1, d_model/2)
    pe[:, 0::2] = torch.sin(position * div_term)
    # Odd indices: cos
    pe[:, 1::2] = torch.cos(position * div_term)

    return pe # (max_len, d_model)
```

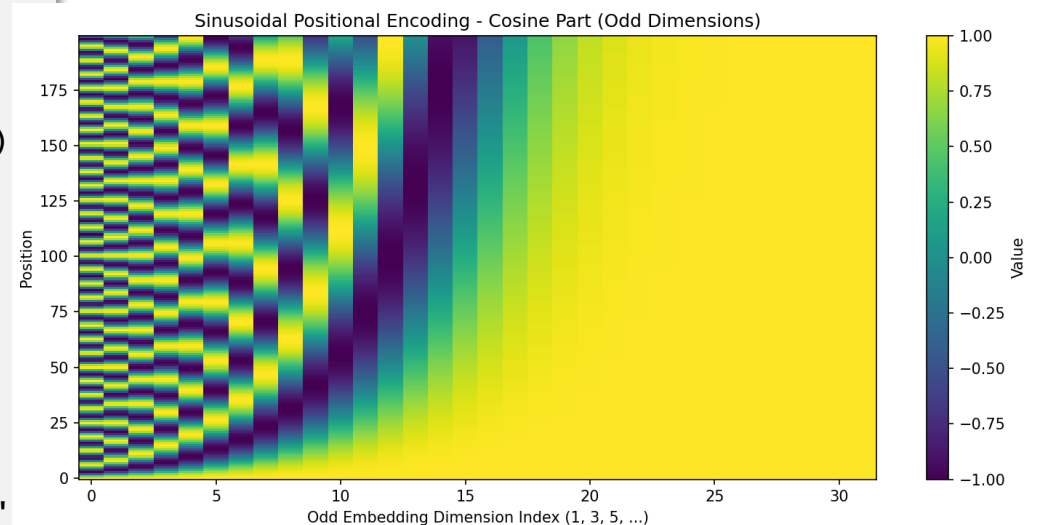
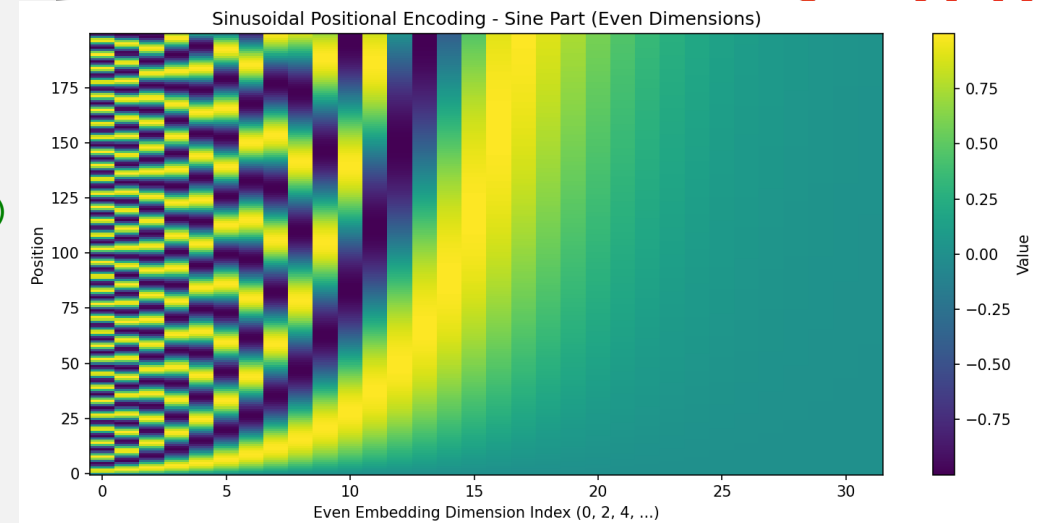
```
# =====
# Visualization: separate heatmaps for sin and cos
# =====

d_model = 64 # embedding dimension
max_len = 200 # number of positions
pe = generate_positional_encoding(d_model, max_len) # (max_len, d_model)

# Extract even (sin) and odd (cos) dimensions
# sin_part: shape (max_len, d_model/2)
# cos_part: shape (max_len, d_model/2)
sin_part = pe[:, 0::2]
cos_part = pe[:, 1::2]

# ----- Heatmap 1: Sine components (even dimensions) -----
plt.figure(figsize=(10, 5))
plt.imshow(sin_part, aspect="auto", origin="lower", cmap="viridis")
plt.colorbar(label="Value")
plt.xlabel("Even Embedding Dimension Index (0, 2, 4, ...)")
plt.ylabel("Position")
plt.title("Sinusoidal Positional Encoding - Sine Part (Even Dimensions)")
plt.tight_layout()

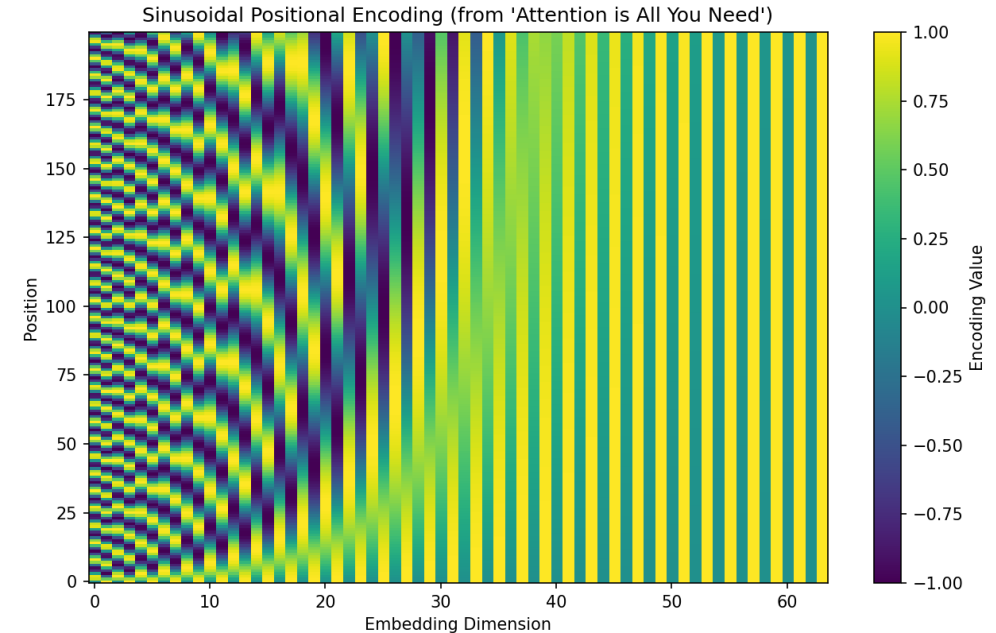
# ----- Heatmap 2: Cosine components (odd dimensions) -----
plt.figure(figsize=(10, 5))
plt.imshow(cos_part, aspect="auto", origin="lower", cmap="viridis")
plt.colorbar(label="Value")
plt.xlabel("Odd Embedding Dimension Index (1, 3, 5, ...)")
plt.ylabel("Position")
plt.title("Sinusoidal Positional Encoding - Cosine Part (Odd Dimensions)")
plt.tight_layout()
plt.show()
```



Positional Encoding (2) : heatmap that combines sin and cos

```
# =====  
# Visualization  
# =====  
  
d_model = 64 # number of embedding dimensions (x-axis)  
max_len = 200 # number of positions (y-axis)  
  
pe = generate_positional_encoding(d_model, max_len) # shape = (200, 64)  
  
plt.figure(figsize=(10, 6))  
  
# imshow expects (rows, cols) = (positions, dimensions)  
plt.imshow(pe, cmap="viridis", aspect="auto", origin="lower")  
  
plt.xlabel("Embedding Dimension")  
plt.ylabel("Position")  
plt.title("Sinusoidal Positional Encoding (from 'Attention is All You Need')")  
plt.colorbar(label="Encoding Value")  
  
plt.show()
```

pe_1.py



```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model, max_len=5000):
        super().__init__()

        # pe: (max_len, d_model)
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float32).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2, dtype=torch.float32)
            * (-math.log(10000.0) / d_model)
        )
        # Even indices: sin
        pe[:, 0::2] = torch.sin(position * div_term)
        # Odd indices: cos
        pe[:, 1::2] = torch.cos(position * div_term)

        # Register as buffer so it is saved with the model but not a learnable parameter
        self.register_buffer("pe", pe.unsqueeze(0)) # (1, max_len, d_model)

    def forward(self, x):
        # x: (batch_size, seq_len, d_model)
        seq_len = x.size(1)

        # Add positional encoding (broadcast along batch dimension)
        x = x + self.pe[:, :seq_len, :]
        return x
```

The **scaled dot-product attention** computes a weighted sum of the value vectors. For a set of queries Q , keys K , and values V , attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Explanation:

- $QK^\top$ computes all pairwise dot products between queries and keys.
- The division by $\sqrt{d_k}$ is the *scaling*; it keeps the magnitude of the dot products from growing too large when d_k is large.
- softmax turns each row into a probability distribution over all positions.
- Multiplying by V computes a weighted sum of value vectors for each query.

This allows each token to attend to (i.e. focus on) other tokens in the sequence.


```
def scaled_dot_product_attention(q, k, v, mask=None):  
  
    # q, k, v: (batch, heads, seq_len, head_dim) -> (B, H, L_q, D)  
    # mask: (batch, 1, 1, seq_len_k) or broadcastable shape (optional)  
  
    d_k = q.size(-1)  
  
    # q @ k.T = (B, H, L_q, D) @ (B, H, D, L_k) -> (B, H, L_q, L_k)  
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k) # (B, H, L_q, L_k)  
  
    if mask is not None:  
        # mask == 0 -> masked out  
        scores = scores.masked_fill(mask == 0, float("-inf"))  
  
    attn = F.softmax(scores, dim=-1)  
    out = torch.matmul(attn, v) # (B, H, L_q, head_dim)  
    return out, attn
```

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V.$$

Instead of using a single attention mechanism, Transformers use **multiple heads**.

Each head has its own projection matrices:

$$Q_h = XW_h^Q, \quad K_h = XW_h^K, \quad V_h = XW_h^V,$$

and produces:

$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h).$$

Then all heads are concatenated:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W^O,$$

where W^O is another learned projection matrix.

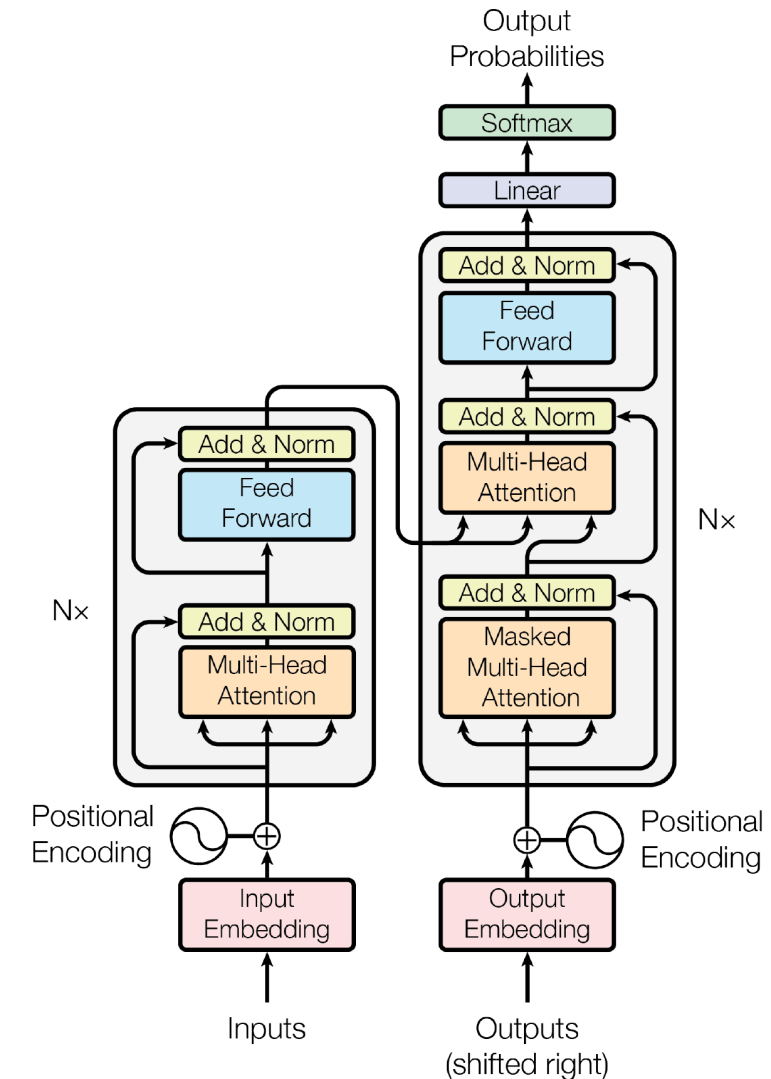
Intuition:

- Each head can focus on different types of relationships (e.g. syntax, long-range dependencies).
- Concatenating them allows the model to combine these different perspectives.

Encoder and Decoder in Transformer

The original Transformer consists of:

- **Encoder:** a stack of identical layers, each containing multi-head self-attention and a position-wise feed-forward network.
- **Decoder:** a stack of layers that uses:
 - masked multi-head self-attention over the output sequence (to prevent looking into the future),
 - multi-head attention over the encoder outputs,
 - a feed-forward network.



Thank you!

